

Aula 12 — Interface e Máquinas de Estado

Teclado matricial, estados finitos e organização do firmware

Microcontroladores — DENE/UFMT

Objetivos da aula

- Explicar a varredura de teclado matricial e justificar a economia de pinos que a motiva.
- Reconhecer a ambiguidade de leitura com múltiplas teclas e sua correção.
- Formalizar comportamento sequencial como máquina de estados finitos e implementá-la de duas formas distintas.
- Projetar a navegação de um menu e a edição de parâmetros sem bloquear o sistema.
- Organizar o firmware em módulos com interfaces explícitas e acoplamento reduzido.

1 Teclado matricial

Um teclado de dezesseis teclas ligadas individualmente consumiria dezesseis pinos — inviável. A organização matricial reduz esse número a oito.

Definição — varredura matricial

Técnica em que as teclas são dispostas na interseção de linhas e colunas. As linhas são acionadas uma de cada vez pelo programa, e as colunas são lidas: a tecla pressionada é identificada pelo par linha ativa e coluna detectada.

Derivação

Para L linhas e C colunas, o número de teclas é $L \cdot C$ e o de pinos é $L + C$. Com quatro por quatro, dezesseis teclas em oito pinos. A economia cresce com o tamanho:

$$\frac{\text{teclas}}{\text{pinos}} = \frac{L \cdot C}{L + C}$$

o que, para uma matriz quadrada de lado k , dá $\frac{k}{2}$ — oito vezes menos pinos num teclado de dezesseis por dezesseis.

Código

```
/* Chamada a cada 5 ms pelo escalonador.
   Retorna a tecla pressionada ou 0xFF se nenhuma. */
uint8_t teclado_varrer(void)
{
    for (uint8_t linha = 0; linha < 4u; linha++) {
        LATB = (uint8_t)(~(1u << linha) & 0x0F); /* ativa uma linha */
        __delay_us(5);                             /* estabiliza */

        uint8_t colunas = (uint8_t)(~(PORTB >> 4) & 0x0F);

        if (colunas != 0) {
            for (uint8_t c = 0; c < 4u; c++) {
```

```
        if (colunas & (1u << c)) {  
            return (uint8_t)(linha * 4u + c);  
        }  
    }  
}  
}  
return 0xFF;  
}
```

Atenção

A varredura exige resistores de elevação nas colunas: sem eles, uma coluna sem tecla pressionada fica flutuante e é lida de forma imprevisível. O microcontrolador oferece resistores internos em uma das portas, habilitados em conjunto — o que costuma decidir em qual porta o teclado é ligado.

Observação

O curto atraso após acionar a linha não é preciosismo: a capacitância das trilhas e do cabo impede que a mudança se propague instantaneamente, e ler cedo demais devolve o estado da linha anterior. É a mesma questão de tempo de estabilização vista na aquisição do conversor.

1.1 Ambiguidade com múltiplas teclas

Atenção

Com duas teclas pressionadas simultaneamente, a matriz cria um caminho elétrico entre linhas, e uma terceira tecla **não pressionada** pode ser detectada — fenômeno conhecido como tecla fantasma. Não há correção por software: a solução é um diodo em série com cada tecla, impedindo o caminho reverso. Em teclados sem diodos, a única saída é **aceitar apenas uma tecla por vez** e descartar leituras múltiplas, que é o que se faz neste projeto.

2 Máquinas de estado finitos

O teclado entrega teclas; o significado de cada uma, porém, depende do que já aconteceu. A mesma tecla confirma um valor num momento e inicia uma edição em outro. Esse tipo de comportamento pede uma formalização.

Definição — máquina de estados finitos

Modelo composto por um conjunto finito de **estados**, um conjunto de **eventos**, uma função de **transição** que determina o próximo estado a partir do estado atual e do evento recebido, e **ações** associadas a transições ou a estados.

Observação

O modelo já apareceu duas vezes no curso sem ser nomeado: o controlador com histerese do encontro 6 é uma máquina de dois estados, e o interpretador de comandos do encontro

11 é uma máquina que consome caracteres. Reconhecer o padrão é o que permite tratá-los com a mesma ferramenta.

2.1 A tabela de transições

Antes de qualquer código, a máquina se projeta numa tabela. Ela é o documento que se revisa, e escrever código sem passar por aqui é a origem da maioria dos menus com comportamento imprevisível.

Estado	Evento	Próximo estado	Ação
Normal	Tecla MENU	Menu	Mostrar primeira opção
Normal	Outra	Normal	Nenhuma
Menu	Cima ou baixo	Menu	Mudar opção exibida
Menu	Tecla OK	Edição	Carregar valor atual
Menu	Tecla VOLTA	Normal	Restaurar tela principal
Edição	Cima ou baixo	Edição	Somar ou subtrair, com limites
Edição	Tecla OK	Normal	Aplicar e persistir
Edição	Tecla VOLTA	Normal	Descartar alteração
Edição	Sem tecla por 30 s	Normal	Descartar por inatividade

Tabela 1: Tabela de transições do menu de ajuste.

Atenção

A última linha é frequentemente esquecida e é a mais importante em equipamentos sem operador permanente. Sem ela, um sistema deixado em modo de edição **permanece nele indefinidamente** — sem controlar a planta, se a edição suspender o controle. Toda máquina de estados com interação humana precisa de saída por tempo em cada estado que não seja o de repouso.

2.2 Implementação

Código

```
typedef enum { EST_NORMAL, EST_MENU, EST_EDICAO } estado_t;

static estado_t estado = EST_NORMAL;
static int16_t editando = 0;
static uint16_t ms_inativo = 0;

void ihm_processar(uint8_t tecla) /* chamada a cada 5 ms */
{
    if (tecla == TECLA_NENHUMA) {
        if (estado != EST_NORMAL && ++ms_inativo >= 6000u) {
            estado = EST_NORMAL; /* 30 s de inatividade */
        }
    }
}
```

```
        ms_inativo = 0;
    }
    return;
}
ms_inativo = 0;

switch (estado) {
case EST_NORMAL:
    if (tecla == TECLA_MENU) {
        estado = EST_MENU;
    }
    break;

case EST_MENU:
    if (tecla == TECLA_OK) {
        editando = alvo_atual();
        estado = EST_EDICAO;
    } else if (tecla == TECLA_VOLTA) {
        estado = EST_NORMAL;
    }
    break;

case EST_EDICAO:
    if (tecla == TECLA_CIMA) {
        editando = limitar(editando + 5, MIN_ALVO, MAX_ALVO);
    } else if (tecla == TECLA_BAIXO) {
        editando = limitar(editando - 5, MIN_ALVO, MAX_ALVO);
    } else if (tecla == TECLA_OK) {
        alvo_definir(editando);
        estado = EST_NORMAL;
    } else if (tecla == TECLA_VOLTA) {
        estado = EST_NORMAL;
    }
    break;
}
}
```

Observação

A função nunca espera. Ela recebe um evento, decide, atualiza o estado e retorna — o mesmo formato de todas as tarefas desde o encontro 7. É o que permite que o controle da planta continue funcionando enquanto o operador navega pelo menu, propriedade que uma implementação com laços de espera perderia imediatamente.

Atenção

Editar uma **cópia** do valor, e não o valor em uso, é decisão de projeto e não detalhe: permite descartar a alteração, e evita que valores intermediários — percorridos enquanto o operador ajusta — cheguem ao controlador. Sem isso, subir de 20 para 80 graus faria a planta perseguir cada valor intermediário.

3 Organização do firmware

Com nove módulos convivendo, a organização deixa de ser questão de gosto.

Prática	Como	Por quê
Interface no cabeçalho	Apenas o que outros módulos usam	O cabeçalho vira a documentação do módulo
Ocultar o interno	<code>static</code> em funções e variáveis não exportadas	Impede acoplamento acidental e libera nomes
Evitar variáveis globais	Acesso por funções do módulo dono	Localiza a responsabilidade por cada dado
Tipos de largura explícita	<code>uint8_t</code> em vez de <code>int</code>	O tamanho de <code>int</code> varia entre plataformas
Tabelas constantes em Flash	<code>const</code>	Preserva os 2 KB de RAM
Uma tarefa, uma função	Sem espera; chamada pelo escalonador	Mantém a arquitetura não bloqueante

Tabela 2: Práticas de organização e sua justificativa.

Observação

A arquitetura completa do projeto pode ser lida em três camadas. Embaixo, os módulos de **acesso ao hardware** — display, conversor, teclado, serial — que conhecem registradores e não conhecem termostatos. No meio, os módulos de **aplicação** — controle, interface, telemetria, persistência — que conhecem o problema e não conhecem registradores. No topo, o **escalonador**, que só sabe chamar tarefas em seus ritmos. Trocar de microcontrolador exigiria reescrever a camada de baixo e preservaria as demais — afirmação que o seminário terá ocasião de testar.

4 Transposição

Máquinas de estado são independentes de plataforma: a mesma tabela de transições vale em qualquer arquitetura. Em sistemas maiores, aparecem duas alternativas — tabelas de transição em memória, percorridas por um interpretador genérico, e sistemas operacionais de tempo real, em que cada tarefa é uma linha de execução independente que pode ser suspensa.

Observação

Vale resistir à tentação de considerar o sistema operacional sempre superior. Ele resolve o problema de tarefas que precisam esperar sem bloquear as demais, ao custo de uma pilha por tarefa, de um escalonador e de toda a disciplina de sincronização — exclusão mútua, semáforos, inversão de prioridade. Para o conjunto de tarefas periódicas e curtas deste projeto, o escalonador cooperativo entrega o mesmo resultado com uma fração da complexidade. **Saber quando o custo se justifica é a competência de projeto; adotá-lo por padrão não é.**

5 Exercícios

Exercício 12.1

Calcule quantos pinos seriam necessários para 24 teclas em ligação individual e em matriz, considerando as configurações de matriz possíveis. Indique qual delas minimiza o número de pinos.

Exercício 12.2

Explique o mecanismo da tecla fantasma com um exemplo concreto de três teclas numa matriz quatro por quatro, indicando o caminho elétrico responsável e onde os diodos devem ser inseridos.

Exercício 12.3

Acrescente à tabela de transições da seção 2 um segundo parâmetro editável — a largura da banda de histerese — sem criar novos estados. Indique que variável adicional é necessária.

Exercício 12.4

A função `ihm_processar` é chamada a cada 5 ms e a saída por inatividade usa o valor 6000. Verifique se esse valor corresponde aos 30 s pretendidos e, se não, corrija-o. Comente o que aconteceria se o intervalo de chamada fosse alterado para 10 ms sem revisar essa constante.

Exercício 12.5

Um colega propõe implementar o menu com laços de espera — “mostra a opção e espera a tecla” — alegando que o código fica mais legível. Avalie a proposta indicando o que exatamente deixaria de funcionar no sistema, e responda se existe algum contexto em que ela seria aceitável.